Governing What AI Agents Can Do

SecureMCP — A Business Overview

Control, proof, and trust for AI agents that take action on business systems.

---

1. Executive Summary

AI assistants used to answer questions. Now they take actions — query a customer database, issue a refund, send an email, update a record, call an internal service. The industry standard that makes this possible is the Model Context Protocol (MCP), and it has been adopted quickly because it works.

What MCP does not include is any notion of permission. It defines how an AI agent discovers what it can do and how it does it. It says nothing about whether the agent *should* — who authorised it, under what limits, on whose behalf, subject to which regulation, and what record survives afterward.

SecureMCP is the layer that adds those answers.

> **MCP gives an AI agent capability. SecureMCP governs that capability.**

In practice, SecureMCP sits between the agent and the systems it can reach, and does four things:

	The question it answers	The business value
Control	Is this action allowed, right now, for this requester?	Actions outside policy do not happen
Proof	What happened, and can it be proven later?	A tamper-evident record an outside auditor can verify
Trust	Should this tool be available at all?	Vetted, signed tools instead of unverified ones
Response	Something looks wrong — now what?	Automatic slowdown, confirmation, or shutdown

It is built as fourteen independent capabilities, each switched on separately. An organisation can start with one capability and expand, rather than committing to an all-or-nothing platform decision. The first deployment can be small.

---

2. Why This Matters Now

Three shifts happened at roughly the same time, and together they created the gap SecureMCP fills.

Agents got access. An AI assistant with no connection to internal systems is limited but safe. An AI assistant connected to a CRM, a billing platform, and a document store is useful — and is now a participant in business processes that previously required a person with a login.

Access became standardised. MCP made connecting an agent to a system routine work rather than a custom integration project. That is why adoption moved fast, and it is also why the number of connected systems per agent is growing rather than shrinking.

Nothing standardised the guardrails. The protocol leaves permission, limits, audit, and oversight entirely to whoever builds the server. In practice that means each team invents its own approach, or defers the question. The result is a familiar pattern: capability arrives first, governance arrives after the first incident.

The exposures this creates are ones any risk committee will recognise:

- An agent takes an action nobody authorised, and there is no record of who or what triggered it
- An agent handles regulated data — health records, payment details, personal data — with no evidence trail that would satisfy an auditor
- A permission granted for one narrow purpose is quietly reused far more broadly
- An agent's behaviour changes — many more calls, unusual targets, higher error rates — and nobody notices until the damage is done
- A tool is installed from outside the organisation with no verification of who wrote it or what it claims to do

None of these are exotic. They are the ordinary consequences of giving software the ability to act without giving it the constraints a person in the same role would have.

---

3. What SecureMCP Is

The most useful comparison is a building with a security desk.

An MCP server is a building full of rooms, each room a system the agent can use. Before SecureMCP, the agent walks in and opens any door it can find. After SecureMCP, there is a desk at the entrance, a badge with specific room access, a visitor log that cannot be quietly edited, and a guard who notices when someone who normally visits two rooms suddenly tries forty.

Three properties define how it behaves.

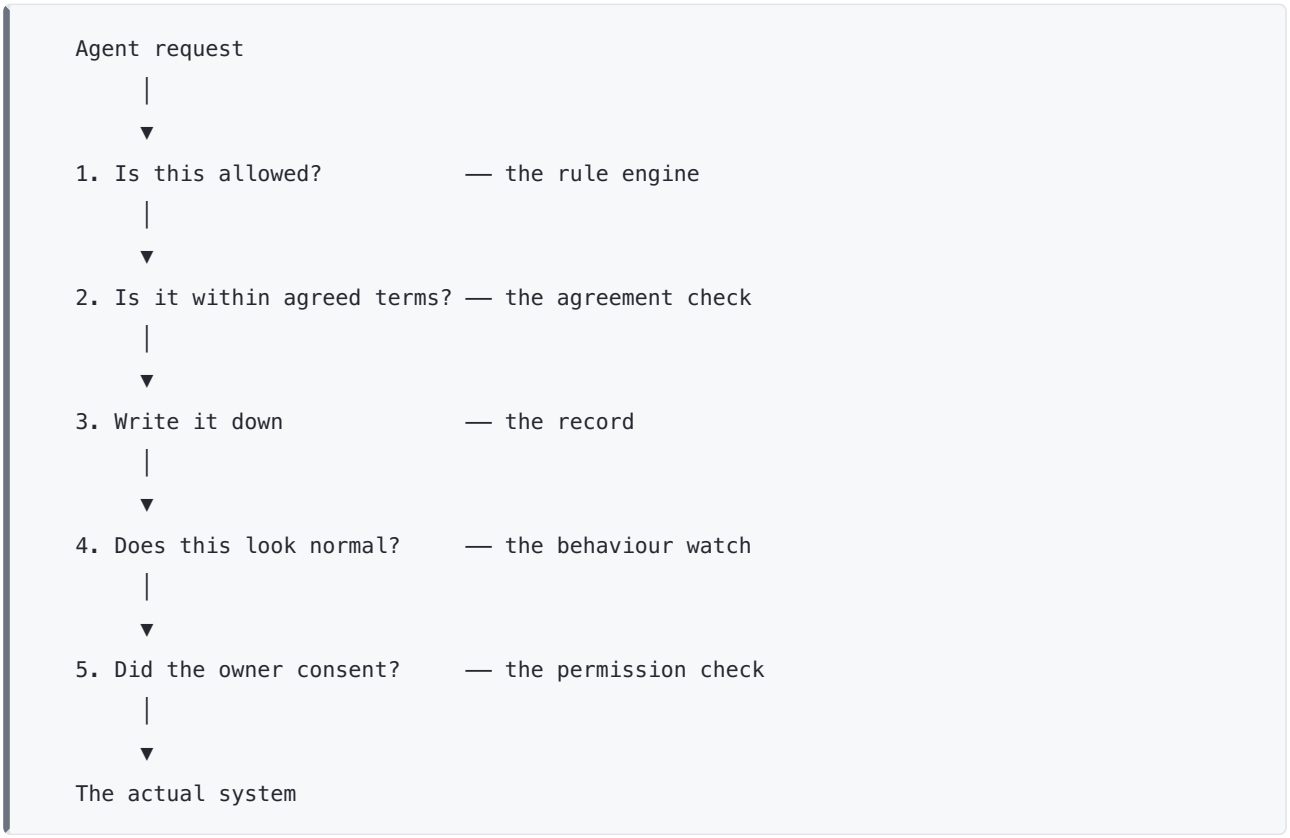
It does not replace what is already there. SecureMCP attaches to an existing MCP server rather than replacing it. Nothing has to be rebuilt, and the server keeps working the way it did. This is the difference between a governance initiative measured in weeks and one measured in quarters.

Every capability is optional and independent. There is no single security mode that either is or is not on. Fourteen capabilities are switched on individually. An organisation concerned with audit turns on record-keeping. One concerned with regulated data turns on the rule engine. Neither requires the other.

When it cannot decide, it blocks. If the rule engine cannot reach a clear answer — a rule component failed, or the request cannot be described properly — the default outcome is to deny the action. This is the correct default for a governance system. The alternative failure mode is the one that produces incidents: a system that quietly allows everything the moment something breaks.

4. How a Request Is Governed

Every request an agent makes passes through five checkpoints in a fixed order before it reaches the system it is trying to use. The order is deliberate — each checkpoint sits where it can do its job most cheaply and most reliably.



Is this allowed? The rule engine evaluates the request against the organisation's policies. It runs first because a clear denial should cost as little as possible.

One consequence is worth drawing out. When an agent asks what it is able to do, SecureMCP filters the answer to only what that requester is actually permitted to use. Tools the agent cannot use do not appear in the list at all. Capability the requester is not entitled to is not merely blocked — it is invisible, which removes the opportunity to probe for it.

Is it within agreed terms? Where an agent has negotiated a working agreement — what it may do, which resources it may touch, for how long — that agreement is checked. Both sides can be required to sign it cryptographically, so neither can later claim they agreed to something different.

Write it down. The action is recorded, successes and failures alike, because a blocked attempt is often the more significant entry. Section 6 covers what this record makes possible.

Does this look normal? SecureMCP builds a picture of each requester's normal behaviour — how often it calls, how fast, how often it errors — and measures each new request against that picture. The principle is the same as credit card fraud detection: the individual transaction is not suspicious, the pattern is.

Responses escalate rather than jumping straight to a block: continue, ask a person to confirm, slow the requester down, or stop it entirely.

Did the owner consent? The final check before the action happens is whether the owner of the data or resource actually permitted this requester to do this thing.

5. The Fourteen Capabilities

Each of these is enabled independently.

Capability	What it does
Rule engine	Decides whether each action is allowed, and records which policy or regulation clause produced the decision
Agreements	Negotiated, signed terms of engagement between the organisation and an agent
Record	A tamper-evident log of everything that happened, verifiable by an outside party
Behaviour watch	Learns normal behaviour per requester and reacts when it changes shape
Permissions	Who has consented to what, including permissions passed from one party to another
Alerts	Emits security events for existing monitoring tools to consume
Certification	Turns a tool's declared behaviour into a signed, verifiable claim
Trust scoring	A single trust score per tool, combining certification, track record, and age
Tool catalogue	The inventory of available tools and their standing
Federation	Lets separate organisations share trust information without giving up their own authority
Revocation list	The recall mechanism — pull a compromised tool out of service immediately
Compliance mapping	Connects controls to named regulatory frameworks for reporting
Sandbox	A declared rulebook for how a tool should behave, plus visibility when it does not (see section 10)
Gateway and dashboard	The read-only view of what the governance layer is doing

Only the first five sit on the path of every request. The rest run when a tool is published, when a question is asked, or on demand.

6. Proof and Audit

Most systems keep logs. Logs have a well-known weakness: whoever controls the system controls the logs. An auditor is being asked to trust the accuracy of a record produced by the party being audited.

SecureMCP's record is built so that trust is not required.

Every entry is linked to the one before it. Think of a numbered ledger where each page carries a fingerprint of the previous page. Changing an old entry breaks every fingerprint after it. Entries cannot be quietly edited, and they cannot be quietly removed.

Any single entry can be proven on its own. A specific action can be proven to belong to the record without handing over the rest of it. A dispute about one transaction does not require disclosing every other transaction to resolve it.

The proof works away from the system. SecureMCP can export a self-contained evidence package for any entry. An auditor, a regulator, or the other side of a dispute can verify it independently, on their own machine, with no access to the running system. Integrity is confirmed by the party who needs the assurance, not asserted by the organisation being examined.

Two points of accuracy. The record captures what was asked and what the outcome was, not a copy of the data that came back — deliberately, so that sensitive results are not duplicated into a permanent store. And each record is anchored to a random starting value unique to it, so a plausible-looking fabricated record cannot be manufactured and passed off as the real one.

The practical test is a question most organisations currently cannot answer: how would the actions taken by an AI agent last March be proven to a regulator today? This is the answer.

---

7. Permissions That Cannot Grow

Permissions in SecureMCP are specific: this requester may do this particular thing with this particular resource, optionally only under certain conditions, and optionally only until a certain date.

Permissions can also be passed along — an agent given access can delegate some of it to another agent, which is necessary in any real multi-agent system. The rule governing that is strict: **a passed-along permission can only ever be narrower than the one it came from.** Nothing can grant more than it was given. Authority shrinks as it spreads; it never grows.

Withdrawal follows the same logic. Revoking a permission automatically revokes everything derived from it. No orphaned access continues working because it was handed off before the withdrawal.

And every decision explains itself. When access is granted through a chain of three delegations, SecureMCP can show that chain. Denials are explained the same way — which missing permission caused it, not merely that something was refused.

Together these address the exposure auditors call privilege creep: access that accumulates quietly until nobody can say why a given system can reach a given dataset.

---

8. Trust in Third-Party Tools

The tools an agent can use are software, often written by someone outside the organisation. Treating them all as equally trustworthy is the same mistake as installing any application a user asks for.

SecureMCP handles this with a chain that will be familiar to anyone who has considered app store review or food safety inspection.

A tool declares what it does. Which resources it needs, and what behaviour to expect.

That declaration is certified and signed. The result is a cryptographic claim that cannot be forged or altered after the fact. Certification comes in levels, from independently audited down to self-declared, and the level is visible.

Unsigned claims count for nothing. By default, a certification without a valid signature cannot improve a tool's standing at all. An unverifiable claim carries no weight inside the system, which is what makes certification meaningful.

Each tool carries a trust score. Three inputs, weighted deliberately:

Input	Weight	Reasoning
Certification	50%	A verified signature is stronger evidence than opinion
Track record	35%	Accumulated behaviour, starting from neutral rather than trusted
Age	15%	Deliberately capped — longevity is weak evidence, and the design refuses to let it become strong evidence

New tools start neutral: neither trusted nor suspected. Scores update immediately when new evidence arrives, and a significant drop raises a high-priority alert.

Compromised tools can be recalled. A revocation list pulls a tool out of service immediately. Where organisations have chosen to federate, a recall can propagate to peers — but each organisation's own list remains authoritative. A peer can inform a recall decision; it cannot make one on another organisation's behalf. That distinction is what makes federation acceptable to security teams who would otherwise decline to participate.

---

9. Compliance and Regulated Industries

Six frameworks are modelled directly: **GDPR, HIPAA, SOC 2, ISO 27001, PCI-DSS, and NIST 800-53.**

The mechanism that makes this more than a checklist: when the rule engine blocks an action, the decision can carry a citation to the specific clause that caused it — the regulation, the article, a link to the primary source, and *which version of the text* was in force. Years later, an audit can reconstruct not only that a rule applied but which wording of it did.

The practical difference shows up in how an audit proceeds. A question such as "how is an AI agent prevented from touching patient data without authorisation?" is normally answered with a document describing intent, followed by weeks of evidence gathering. Here it is answered with a report generated from the system's own records.

One point of precision: SecureMCP helps an organisation implement and evidence controls that map to these frameworks. It does not by itself make an organisation compliant, and it is not a certification. Compliance is an organisational outcome involving process, people, and an auditor.

Where the fit is strongest:

Sector	The pressure	The relevant capabilities
Healthcare	HIPAA; patient data touched by agents	Rule engine with clause citations, record, permissions
Financial services	Audit requirements, transaction traceability	Record with exportable proof, behaviour watch, agreements
Regulated enterprise	Data residency, access review, privilege audits	Permissions that cannot expand, cascading withdrawal, compliance reports
Platforms and marketplaces	Third-party tools of unknown provenance	Certification, trust scoring, revocation, federation
Government and defence	NIST control mapping, verifiable audit	Full stack, with an operating-system sandbox alongside (section 10)

10. Scope and Limitations

A governance layer is only as good as the accuracy of its claims. These are the points where SecureMCP's guarantees stop, stated plainly so that a real boundary can be placed where one is needed.

The sandbox is a rulebook, not a locked room. This is the most important one. A tool declares how it will behave, and SecureMCP checks it against that declaration and records any divergence. But the tool has to participate. A tool deliberately built to ignore the rules is not stopped by this layer.

The correct pairing is to run tools inside a real operating-system sandbox for containment, with SecureMCP as the meaning-aware layer on top — one that understands that a given action touches health records, in production, at high risk. That combination is stronger than either alone.

The agreement check is permissive if it fails. If agreement evaluation itself errors, terms are accepted rather than rejected — the opposite of the rule engine's behaviour. Because the rule engine runs first and does deny on failure, the categorical protections still hold. Agreements should not be relied on as the primary control.

One development setting must never reach production. A configuration option allows unsigned certifications to report as valid. It exists for local development and testing; in production it would produce claims that cannot be checked. It defaults to off and should remain off.

Local desktop mode can be configured to skip enforcement. For local use, where the surrounding application is trusted, enforcement can be bypassed. It is off by default and warns loudly when combined with active rules.

Identity comes from elsewhere. SecureMCP tracks a short identifier to correlate activity across its records. That is a correlation key, not proof of identity. Establishing who is really making a request is the authentication layer's responsibility, and SecureMCP relies on it rather than replacing it.

Some figures are estimates. Usage counts are approximations — useful for spotting unusual volume, not suitable for billing.

Trust scores need persistent storage to survive a restart. With storage configured they persist; without it they are rebuilt.

One advanced configuration has a known gap. When federation and shared permissions are used together, the current start-up sequence does not connect the two as intended. It is documented, understood, and has a straightforward fix.

11. A Phased Adoption Path

Because every capability is independent, adoption does not require a single platform decision.

Stage 1 — Establish visibility. Turn on the record. Nothing is blocked and nothing breaks, and the result is a verifiable log of every action the organisation's agents take. No behaviour changes, and the audit value is immediate. The first look at what agents are actually doing is usually itself informative.

Stage 2 — Add the rules. With visibility established, policy can be written against observed reality rather than speculation. Rules can be tested in simulation against real recorded scenarios before going live, which answers the natural concern that a new rule might block something important.

Stage 3 — Add permissions and behaviour monitoring. Owner consent for sensitive resources, plus automatic response when behaviour changes shape.

Stage 4 — Add trust and certification. Relevant once the number of tools grows, or once third-party tools are in use.

Each stage delivers value on its own. None requires the next.

12. Glossary

Term	Plain meaning
MCP	Model Context Protocol — the industry standard for connecting AI agents to tools and data
Agent	An AI system that takes actions, not just answers questions
Tool	A single capability exposed to an agent — look up a customer, send an email, run a query
Policy	A rule about what is allowed, under what conditions
Fail closed	When the system cannot decide, it denies. The safe default
Tamper-evident	Alteration cannot be prevented, but it cannot be hidden either
Attestation	A signed, verifiable claim about what a tool does
Revocation	Pulling a tool out of service — a recall notice
Delegation	Passing part of a permission to another party
Drift	Behaviour moving away from its established normal pattern
Federation	Separate organisations sharing trust information while each keeps its own authority
Provenance	The verifiable record of what happened

13. Common Questions

Does this slow things down? Only five of the fourteen capabilities run on each request, and the first thing that happens is the cheapest possible rejection. The honest answer depends on how much policy is written, and is best measured in the environment where it will run.

Does the existing MCP server have to be rebuilt? No. SecureMCP attaches to a server that already exists. That is a core design commitment, not a convenience.

What happens if the governance layer itself fails? The rule engine denies rather than allowing. Availability is affected before enforcement is. For a governance system this is the correct trade, and it is the opposite of how most software fails.

Can a single capability be used on its own? Yes, and it is the recommended way to start. See section 11.

How is this different from an API gateway or existing access controls? Those govern network calls and user sessions. SecureMCP governs agent actions with awareness of what the action means — that this particular tool touches health records, in production, at high risk. It also adds what a gateway does not: verifiable proof, behaviour monitoring for each agent, tool certification, and consent that cannot expand as it is delegated.

Does it work with existing monitoring? Yes. Security events are emitted for existing tooling to consume, and there is a read-only API and dashboard.

Where does the data live? In the organisation's own storage. Options range from in-memory for development to SQLite and PostgreSQL for production. Nothing is sent anywhere by default.

Is there anything it should not be relied on for? Three things. The sandbox is not containment — pair it with real isolation. The development-only signing bypass must not be enabled in production. And agreements should not serve as the primary control, since the rule engine is the layer that fails closed.

---

14. Summary

MCP gave AI agents the ability to act. Nothing in the protocol governs whether they should.

SecureMCP adds that governance: rules that decide what is allowed and deny when uncertain, a record an outside party can verify without trusting the system that produced it, permissions that shrink rather than grow as they are shared, trust that must be earned through verifiable signatures, and automatic response when an agent's behaviour changes shape.

It attaches to what already exists, it turns on one capability at a time, and it states exactly where its guarantees stop — which is what makes the guarantees it does make worth relying on.